Spring MVC Security - Custom Login Form - Overview

In this lesson, we are going to explore how to create a custom login form with Spring Security. This is an essential aspect of web development, allowing us to create personalized user experiences. Let's get started!

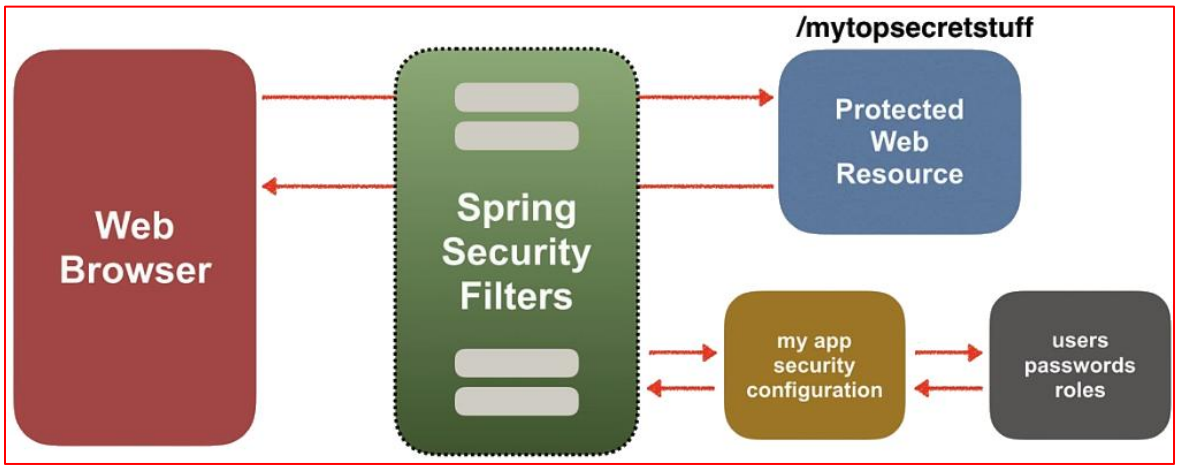
Part I: Introduction to Spring Security and Custom Login Forms

A. The Need for Customization

- By default, Spring provides us with a login form that's ready to use.
- This is great for a quick start but may not be ideal for every situation.
- To better control the appearance and functionality, we'll create our custom login form, allowing us to utilize HTML, CSS, and even incorporate branding like our company logo.

B. Understanding Spring Security

- Spring Security uses filters to pre-process and post-process requests, essentially managing security within the application.
- These filters control access to protected web resources and handle authentication.

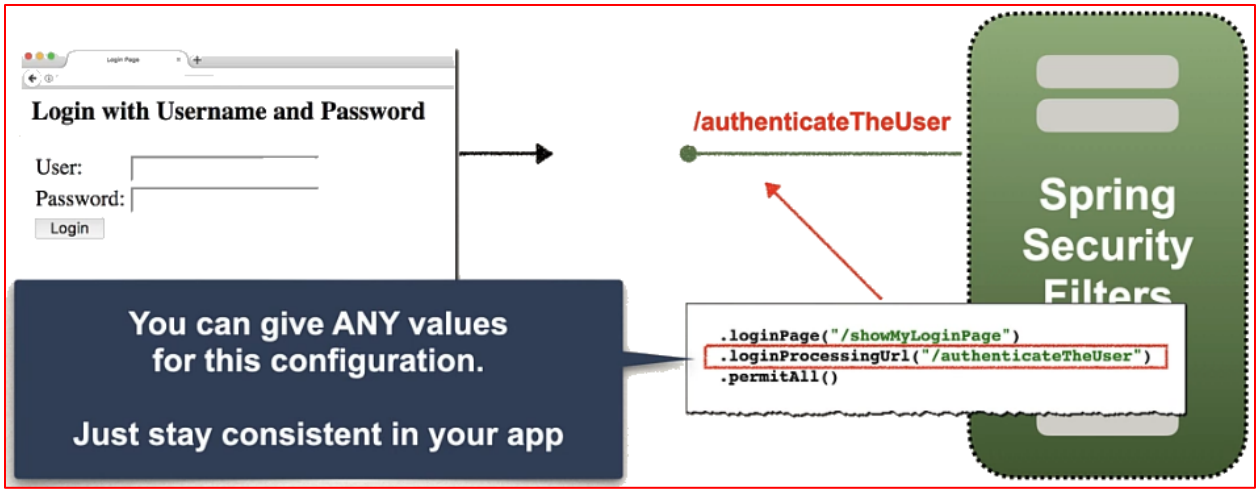


C. Custom Login Form Configuration

- We'll configure Spring to utilize our custom login form instead of the default one.

Part II: The Development Process

At this stage, we will send data to the login processing URL `/authenticateTheUser`. This action is based on the configuration information we previously established in the Spring Security configuration file. Note that you have the freedom to assign any values for this configuration. As we stressed earlier, consistency in your application is key.



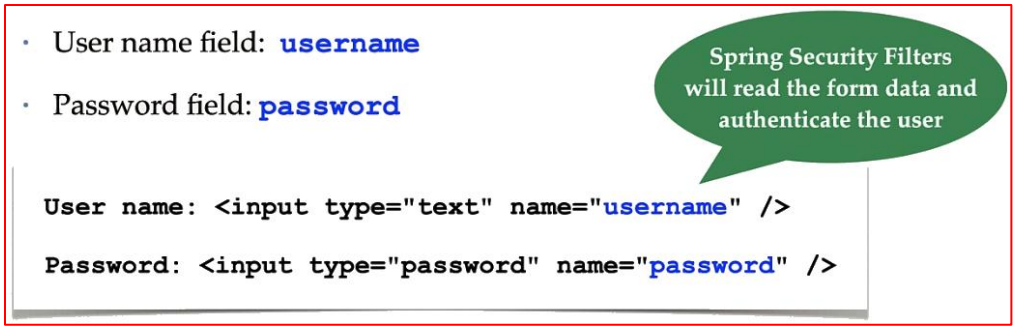
The login processing URL has already been set up and will be handled by the Spring Security filters. One of the appealing aspects of this approach is that you get this functionality for free—there's no additional coding required. This is an example of Spring Security's robust capabilities, which many developers often refer to as "magic."

Reiterating, the URL `/authenticateTheUser` will be managed by Spring. It will process the user ID and password, verifying them as necessary. In constructing the form, when we send data to the login processing URL `/authenticateTheUser`, the action should be specified as `/authenticateTheUser`, and we must POST the data. This is a requirement of the process.

The rationale behind sending it to `/authenticateTheUser` is grounded in the configuration we established in the previous segment of this course. It's worth noting that Spring Security defines some default names for the login form fields. Specifically, the *username* field should be named `username`, and the *password* field should be `password`. These are the standard naming conventions.

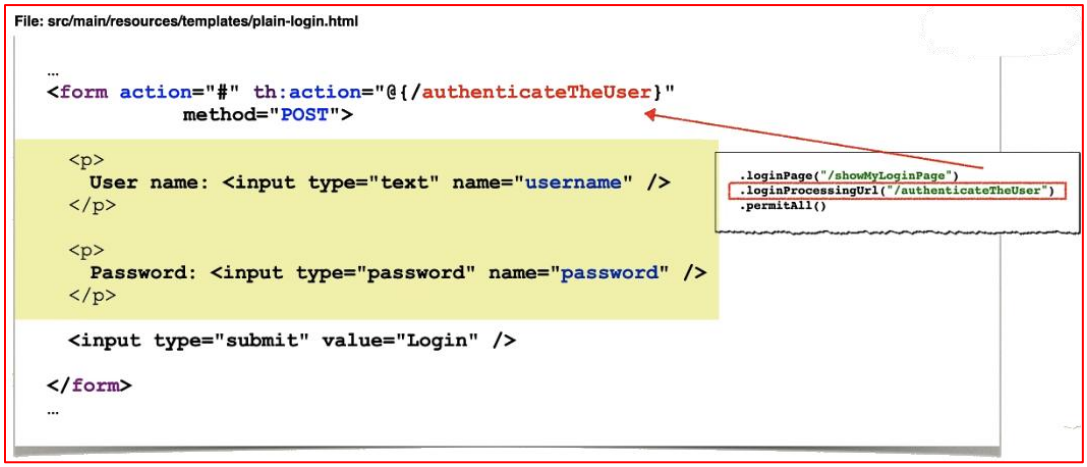


When creating your form, you'll need to set up an input with `type="text"` and `name="username"`, and for the password input, `type="password"` and `name="password"`. Spring Security will read this form data and utilize it to authenticate the user. Spring will handle all authentication and verification in the background, provided that you've used the expected form field names.

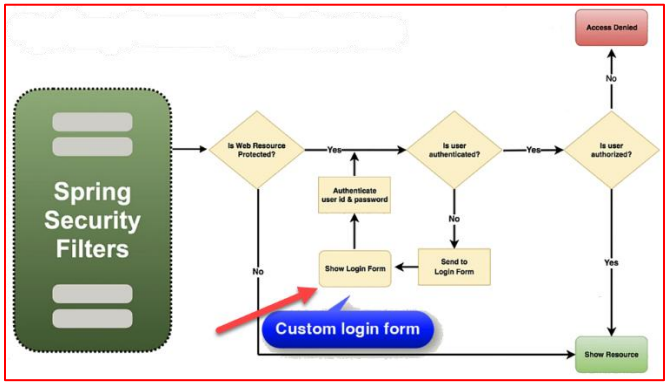


Let's synthesize all this information and create the form. Set `action="/authenticateTheUser"`, and remember that when the form submits the data, it will utilize the login processing URL. You can then insert the fields for username and password, followed by the submit button.

The authentication will actually be processed by the Spring Security filters in the background, validating the username and password.

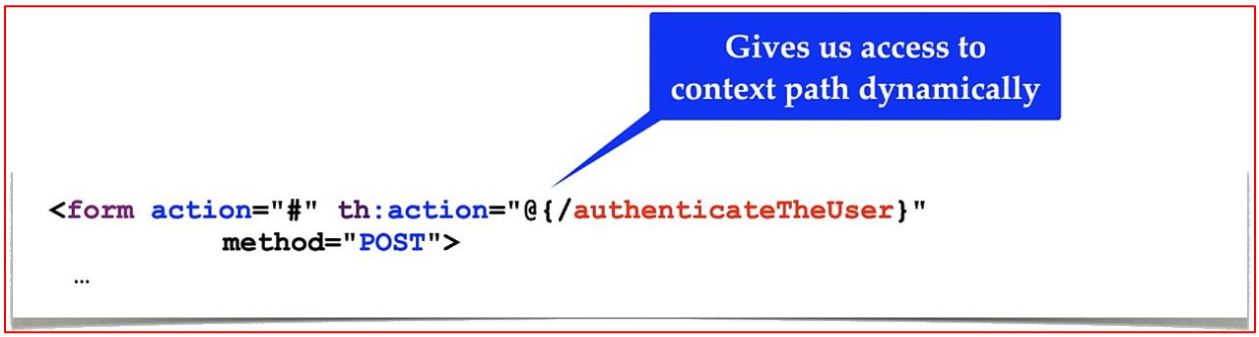


Moving to the background processes, when using Spring Security, the security filters read the information and authenticate the user IDs and passwords based on the particular authentication manager you have in place. This could be in-memory, database, LDAP, and so on. We've navigated through the process of incorporating our custom login form to enhance and personalize the login experience.



Now, let's touch briefly on the **context path**. We introduced some code earlier without a detailed explanation, and you might be wondering about '@' and the terms "context root" and "context path." (Essentially, these terms are interchangeable.)

So, what's the benefit of this approach? It allows us to dynamically access the context path, making it available for any links, such as form submissions, HREF links, etc., within our application. Using the context path in this manner enables us to keep links relative to the application's context path. If you ever change the context path of your application, the links will still function properly.



Avoiding the hard-coding of the context path is a widely accepted best practice in the industry. We highly recommend that you implement this approach in your applications.

A Quick Reminder About Context Path in Spring

The context path refers to a URL pattern that determines which parts of your application are handled by which servlets. In a Spring application, the context path is a prefix added to the URLs of all the application's servlets, effectively grouping them into one logical group or "context."

For example, if your Spring application is configured with a context path of `/app`, then all the application's URLs would start with `/app`. So, a servlet that's mapped to the URL pattern `/login` would actually be accessible at `/app/login`.

The context path is useful when you want to host multiple applications on a single server and need to differentiate between them. By setting different context paths for different applications, the server can determine which application should handle a given request.

In Spring Boot, you can configure the context path by setting the `server.servlet.context-path` property in your `application.properties` or `application.yml` file. Here's an example:

```
server.servlet.context-path=/app
```

This will prefix all your application's URLs with `/app`, as described above.